Python Programmation

CM 6 : Python Libraries

Julien Bezançon

julien.bezancon@univ-lorraine.fr / `https://julienbez.github.io`

Institut des sciences du Digital Management & Cognition

COMPOSANTE DE L'UNIVERSITÉ DE LORRAINE

Loria

1 Before we start...

2 Libraries

3 Environments

4 Modules

What did you learn during last lecture ?
Any questions ?

Last Lecture Doggy bag

File handling

- Open and write a file
- Encoding and pathing
- String parsing
- Dedicated libraries

File navigation

- File VS python structures
- Common errors



1 Before we start...

2 Libraries

3 Environments

4 Modules

What is a Library ?

```
import os
import json
import glob

import pandas as pd
from tqdm import tqdm
```

Libraries are the toolboxes of Python

- Contains pre-written code and functions
- Can be installed and added freely
- Designed for specific usecases

An Example: the "os" Library

```
import os

# some calls from the os Library
os.walk()
os.rename()
os.mkdir()
os.exit()
```

"os" is a standard library

- Installed by default with Python
- Other libraries need to be manually added

Most libraries are not Installed by Default

```
import pandas as pd
df = pd.read_csv('data.csv')
```

```
> python my_script.py
Traceback (most recent call last):
  File "my_script.py", line 1, in <module>
    import pandas
ImportError:  No module named pandas
```

How to Install a Library

With pip

```
pip install pandas
```

With uv

```
uv add pandas
```

With conda

```
conda install pandas
```

Installing a List of Libraries

With pip

```
pip install -r requirements.txt
```

With uv

```
uv add -r requirements.txt
```

With conda

```
conda create --file environment.yml
```

We will now overview 3 packages.
Download the jupyter-notebooks on Arche.

Packages: Summary

Python has hundred of thousands of packages

- We will not see them all in this lecture
- You will not remember how to use each one of them

Best practices

- Get used to python package documentation and common usages
- Do not hesitate to use forums and search engines ($\neq$ **generative AI !!!**)

You will master Python by developping good habits.
Number one good habit = knowing how to find solutions.

1 Before we start...

2 Libraries

3 Environments

4 Modules

Common Issues in Python

Library versions issues

- One script needs pandas v2.2.3 and the other needs pandas v3.0.6
- You can only have one version of a library

Python versions issues

- You want to use an old script, but it uses Python 2.7
- You can have only one version of Python installed

Solution: create Python virtual environments !

What is a Python Virtual Environment ?

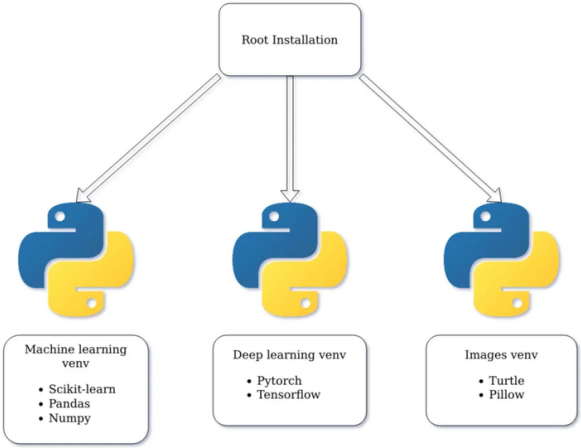
From W3School[1]

> A virtual environment in Python is an isolated environment on your computer, where you can run and test your Python projects.

Python virtual environments (venv) allow you to manage projects which have different Python configurations.

[1] `https://www.w3schools.com/python/python_virtualenv.asp`

How does a venv Works ?



Source: `https://www.python-engineer.com/`

Venv Cheatsheet (python + pip)

How to create a venv

```
> python -m venv venv_name
> source venv_name/bin/activate
```

How to use jupyter-notebook with this venv

```
> pip install --user ipykernel
> python -m ipykernel install --user
```

Venv Cheatsheet (uv)

How to create a venv

```
> cd my_project
> uv init
```

How to use jupyter-notebook with this venv

```
> uv add jupyter
> uv run jupyter notebook
```

Venv Cheatsheet (conda)

How to create a venv

```
> conda create --name venv_name
> conda activate venv_name
```

How to use jupyter-notebook with this venv

```
> conda install jupyter
> jupyter notebook
```

Using the Default environment is BAD

Library compatibilities issues

- Two libraries have common dependencies - but in different versions
- You WILL end up re-installing everything

Reproductibility issues

- You want to upload your code with mandatories libraries
- You want to list each library and the version used - as well as their dependencies

Most of the time, you won't even realize there is an issue !

1 Before we start...

2 Libraries

3 Environments

4 Modules

What is a Module ?

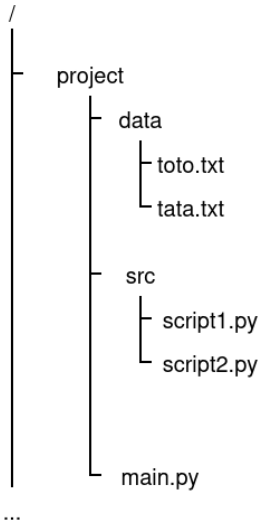
```
from script_1 import function_1, function_2
from src.others.script_2 import *
import script_3
import script_3 as sc

function_1()
function_2()
script_3.function_3()
sc.function_3()
```

Modules help you to structure your projects

- They must be a python file (.py extension)
- They should only contain functions !

How to use Modules ?



Good practices

- Keep modules short and specific
- Place modules in a child folder
- Add `__init__.py` in said child folder

- Avoid circular module imports
- Import everything in a "main" script

Some Questions

What happens if...

- I import script1 in script2 and script2 in script1 ?
- I import script1, which contains package imports ?
- I import script1, which contains variable names ?

- I want to access toto.txt from script1 ?
- I want to access toto.txt from script1 from main ?

Always be careful with paths when using modules !

Doggy bag

Libraries

- The toolboxes of python
- pandas, sklearn and matplotlib

Virtual env

- Use them !
- Global venv = BAD

Modules

- Snippets of code
- Better code structure

